

Version control & Git

Nguyễn Thanh Quân - ntquan@fit.hcmus.edu.vn

Agenda

— — —

1. Introduction to Version Control Systems
2. Introduction to Git
3. Introduction to GitHub
4. Git Merge vs Rebase
5. Branching strategy

1. Introduction to Version Control Systems

A Version Control System (VCS) is a tool that helps track changes in source code or any files over time. This allows multiple people to work on a project without worrying about conflicts and makes it easy to revert to a previous version when needed.

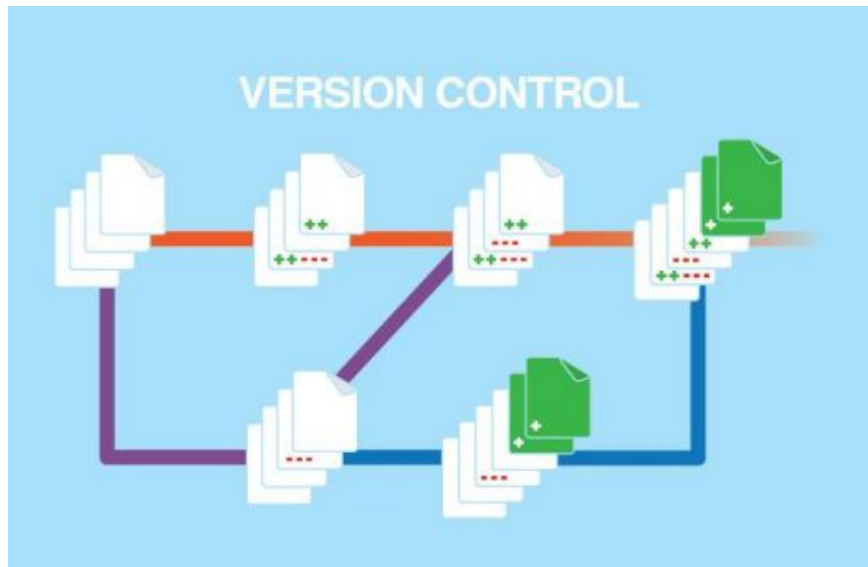


1. Introduction to Version Control Systems

— — —

Why is VCS needed?

- **Tracking changes:** VCS stores all old and new versions of files, helping to monitor the development process of a project.
- **Code recovery:** Allows users to revert to previous versions if errors occur in the current version.
- **Effective collaboration:** Team members can work on the same project without worrying about code conflicts or data loss.
- **Branch management:** VCS allows the creation of multiple branches to develop different features in parallel and merge them when needed.



1. Introduction to Version Control Systems

— — —

Types of Version Control Systems (VCS):

- Local Version Control Systems
 - Stores versions of files on a local machine.
 - Simple but not suitable for team collaboration.
 - Example: RCS (Revision Control System).
- Centralized Version Control Systems (CVCS)
 - Uses a central server to store all versions.
 - Developers pull and push changes from/to the central repository.
 - Single point of failure: if the server crashes, data may be lost.
 - Examples: SVN (Apache Subversion), Perforce.
- Distributed Version Control Systems (DVCS)
 - Every developer has a complete copy of the repository.
 - Enables offline work and reduces dependency on a central server.
 - More robust against data loss.
 - Examples: Git, Mercurial.

2. Introduction to Git

What is Git?

- Git is a free and widely used distributed version control system, developed by Linus Torvalds in 2005. It is designed for easy source code management and supports team collaboration in software development.
- Git allows tracking the history of project changes, managing branches, and merging efficiently.



2. Introduction to Git

— — —

Basic Concepts in Git:

- **Repository (repo):** A storage location that contains all source code files and the change history of a project.
 - **Local Repository**
 - **Remote Repository**
- **Commit:** Saves a version of the source code after making changes.
- **Branch:** Allows developing new features or fixing bugs without affecting the main codebase.
- **Merge:** Combines changes from one branch into another.
- **Clone:** Copies a repository from the server to a local computer.

2. Introduction to Git

— — —

Local Repository

- A Local Repository is a version of the repository stored on a user's personal computer.
- Each person working with Git has a clone of the repository on their local machine, allowing them to work offline and commit changes before syncing with a remote repository.
- Common Commands in Local Repository:
 - `git init` - Initialize a new Git repository.
 - `git clone <repository_url>` - Clone an existing repository to your local machine.
 - `git status` - Check the status of files in the working directory.
 - `git add <file>` - Stage a specific file for commit.
 - `git add .` - Stage all modified files for commit.
 - `git commit -m "Message"` - Commit changes with a message.
 - `git log` - View commit history.
 - `git diff` - Show differences between modified files and the last commit.
 - `git checkout <branch>` - Switch to another branch.
 - `git branch` - List all branches in the repository.

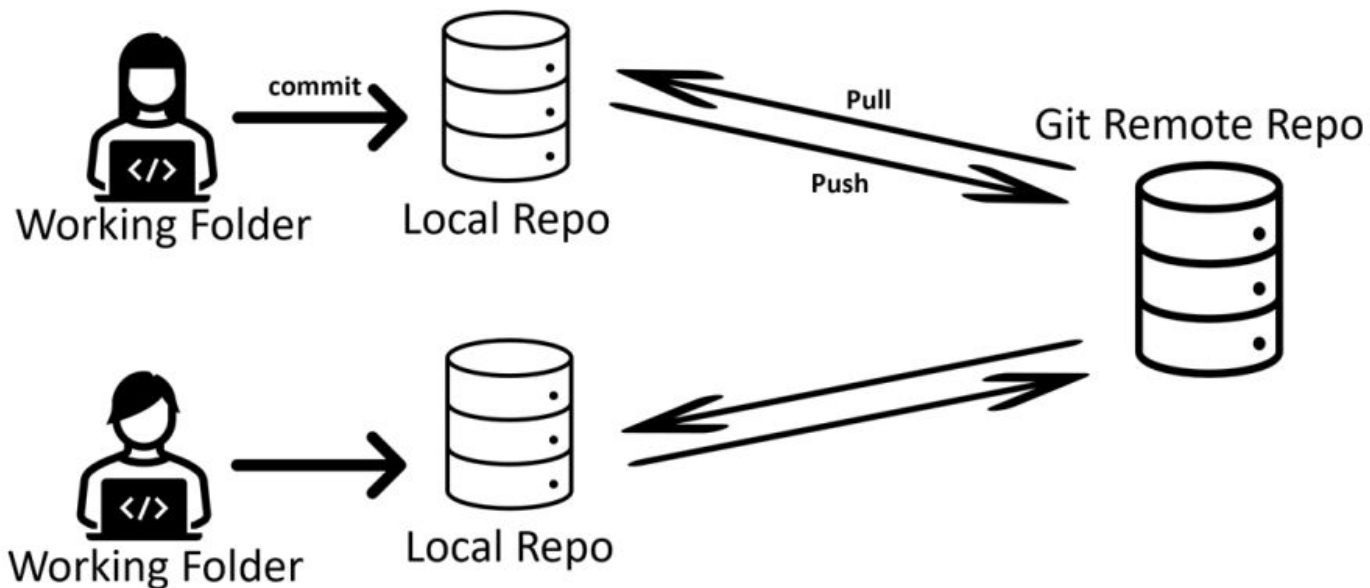
2. Introduction to Git

Remote Repository

- A Remote Repository is a version of the repository stored on a remote server (such as GitHub, GitLab, or Bitbucket).
- It enables multiple users to collaborate on the same project, share, and manage source code remotely.
- Common Commands for Working with a Remote Repository:
 - `git remote add <name> <repository_url>` - Add a remote repository.
 - `git remote -v` - List remote repositories.
 - `git pull <remote> <branch>` - Fetch and merge changes from a remote branch.
 - `git push <remote> <branch>` - Push local changes to a remote repository.
 - `git fetch <remote>` - Fetch updates from the remote repository without merging.
 - `git merge <branch>` - Merge fetched changes into the current branch.
 - `git push -u <remote> <branch>` - Push changes and set the upstream branch.
 - `git clone <repository_url>` - Clone a remote repository to your local machine.

2. Introduction to Git

Remote Repository vs Local Repository



3. Introduction to GitHub

What is GitHub?

- GitHub is a web-based platform for hosting and managing source code using the Git version control system. It allows developers to store code, track changes, and collaborate with others on the same project.
- GitHub provides an intuitive interface and powerful tools to support source code management and teamwork, helping teams work more efficiently.



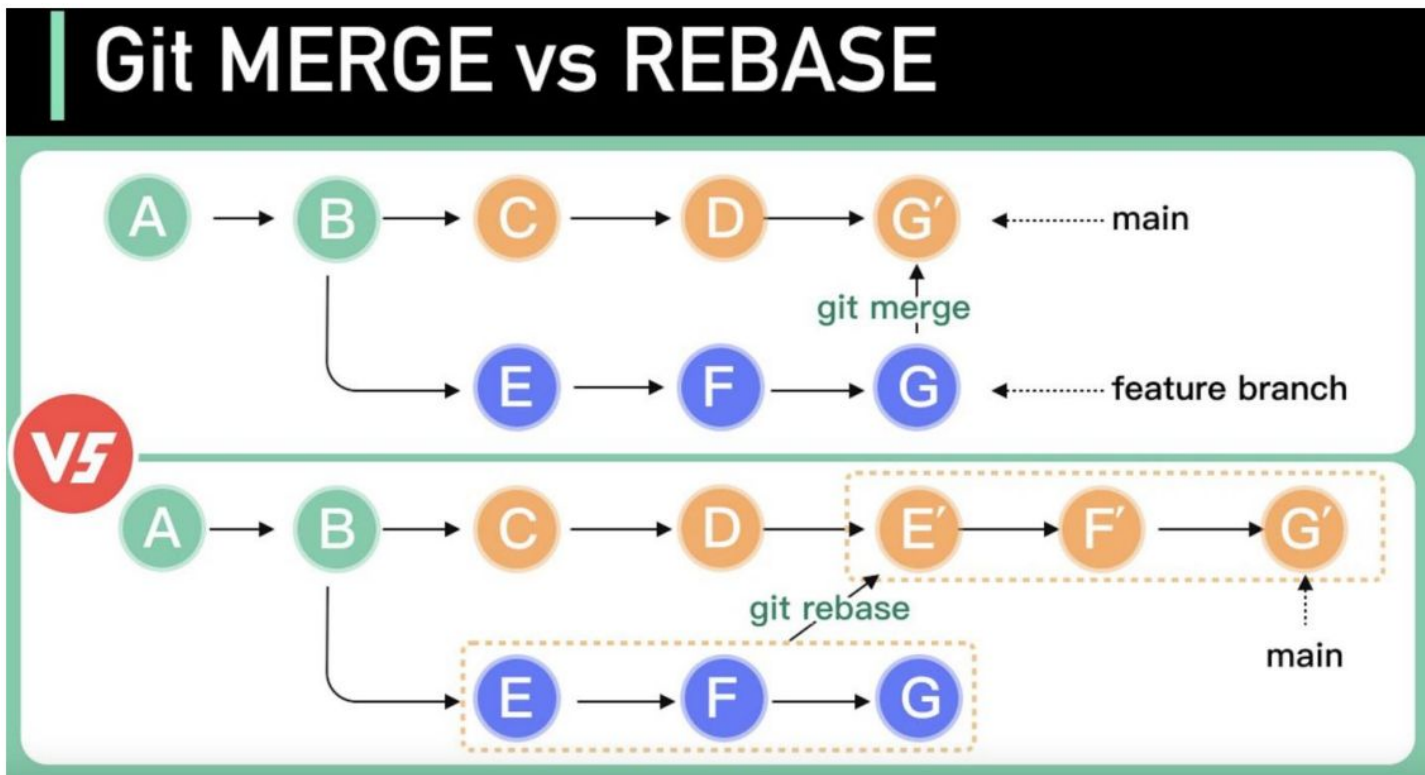
3. Introduction to GitHub

Each repository on GitHub can be either public (accessible to everyone) or private (restricted access), depending on the project's purpose and requirements.

Key Features of a GitHub Repository:

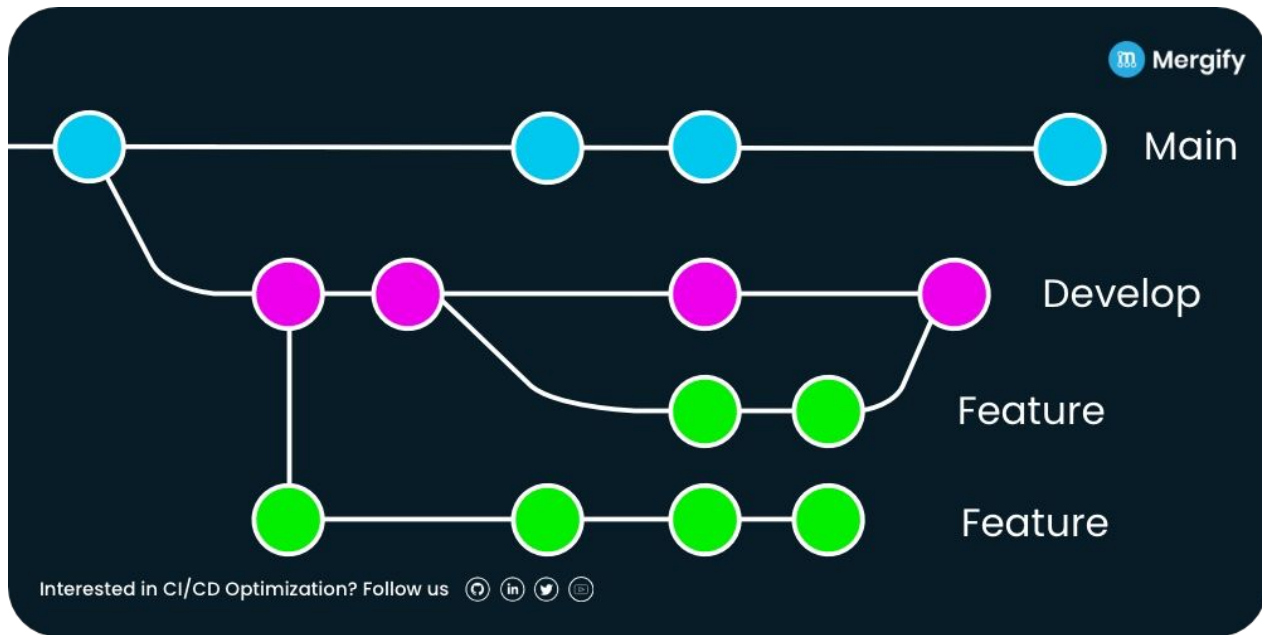
- **Code:** Stores the project's source code.
- **Commits:** Tracks the history of changes in the repository.
- **Branches:** Manages parallel development versions of the code.
- **Pull Requests:** A tool for submitting merge requests from a feature branch to the main branch.
- **Issues:** Used for tracking bugs, feature requests, or project tasks.

4. Git Merge vs Rebase



5. Branching strategy

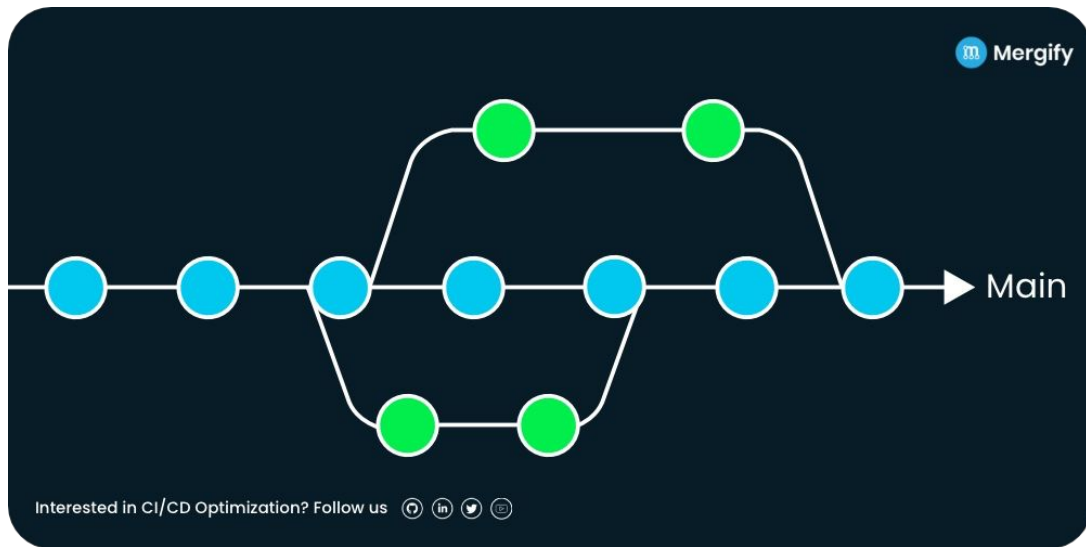
- Gitflow



<https://blog.mergify.com>

5. Branching strategy

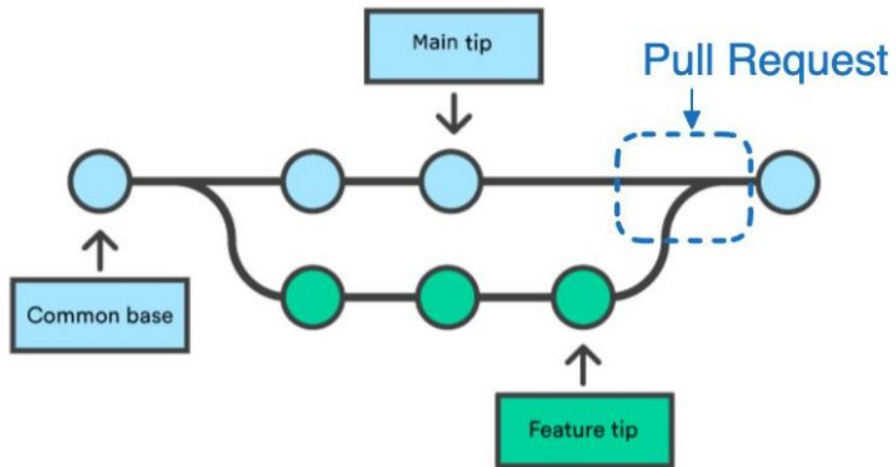
- Trunk-Based Development



<https://blog.mergify.com>

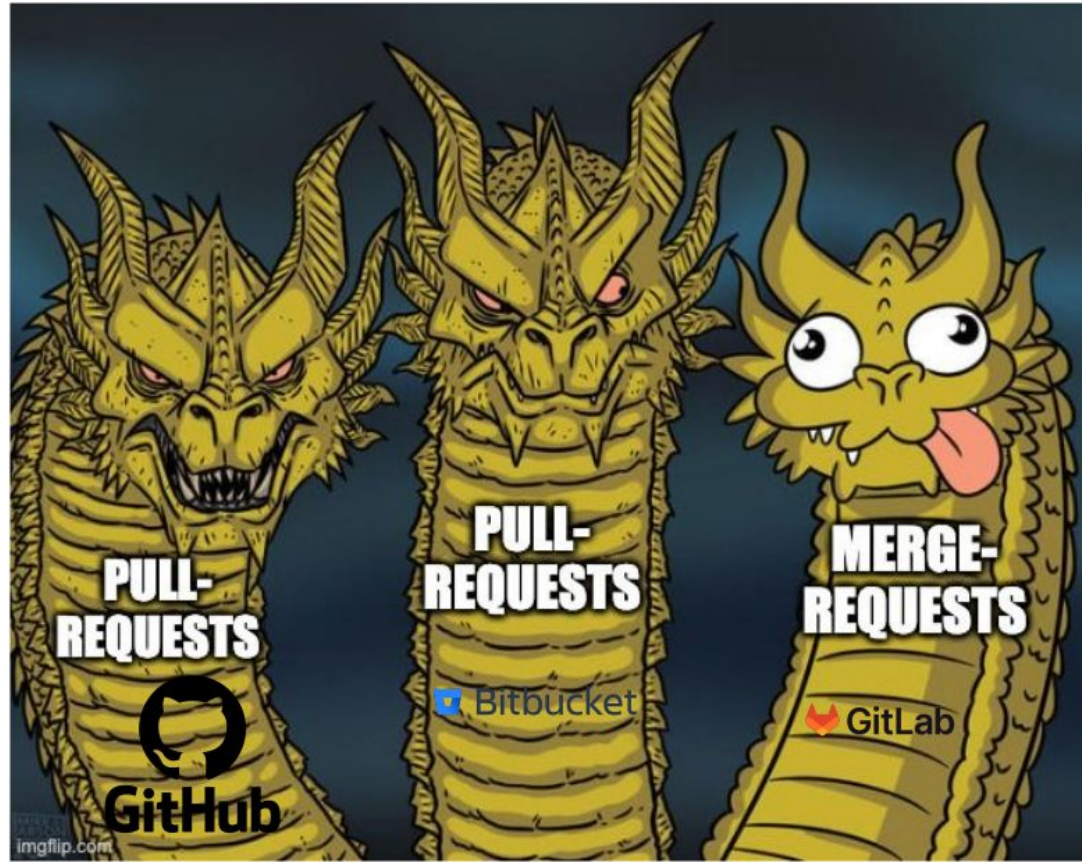
Pull requests

- **Pull Request** (aka **Merge Request**) $\Rightarrow$ Process to merge branch into main
- Select members that need to **review the codes** before the branch is merged to main.



What is the correct name...

Pull request == Merge
request → Synonyms



Pull requests: Code review

Why it is important

- Benefits of code review:
 - **Quality Assurance:** Is this actually the best implementation?
 - **Knowledge sharing:** Aligned team on how the whole code base works.
 - **Component alignment:** Possible catch of logic flaws outside of the feature being reviewed.
 - **Coding standards:** Consistent over code base
 - **Mentorship:** Guidance for less experienced developers
 - **Documentation:** Guarantees that codes are readable
 - **Team morale:** We work as a group, less isolated features
- Tech-lead needs to dedicate time to code review

Pull requests: Code review

— — —

What reviewers should look at

- **Design:** Is the code well-designed and appropriate for your system?
- **Functionality:** Does the code behave as the author likely intended? Is the way the code behaves good for its users?
- **Complexity:** Could the code be made simpler? Would another developer be able to easily understand and use this code when they come across it in the future?
- **Tests:** Does the code have correct and well-designed automated tests?
- **Naming:** Did the developer choose clear names for variables, classes, methods, etc.?
- **Comments:** Are the comments clear and useful?
- **Style:** Does the code follow our style guides?
- **Documentation:** Did the developer also update relevant documentation?

Importance of documentation



Monorepo vs Multi-repo

— — —



Payment



Shopping cart



Inventory

Monorepo vs Multi-repo

— — —

- Monorepo
 - Slow git pull
 - Deploy coupled
 - Hard to grant limited granular git access

```
├── packages
│   ├── utils
│   │   └── package.json
│   ├── components
│   │   ├── src
│   │   └── package.json
│   ├── services
│   │   └── package.json
│   └── ...
├── payment
│   ├── components
│   ├── pages
│   │   └── package.json
│   ├── node_modules
│   ├── webpack.config.js
│   ├── tsconfig.json
│   └── README.md
├── shopping-cart
│   ├── components
│   ├── pages
│   │   └── package.json
│   ├── node_modules
│   ├── webpack.config.js
│   ├── tsconfig.json
│   └── README.md
├── inventory
│   ├── components
│   ├── pages
│   │   └── package.json
│   ├── node_modules
│   ├── webpack.config.js
│   ├── tsconfig.json
│   └── README.md
├── node_modules
├── package.json
└── README.md
```

Monorepo vs Multi-repo

— — —

- Multi-repo
 - Hard to share resources(components, configs, utils and test suites) - submodule
 - Publish the utils repo, component repo and test suites repo separately, and manually manage the versions of these dependencies

```
// Repo1: Payment
├─ assets
├─ components
│   └─ PaymentButton.js
├─ node_modules
├─ pages
│   └─ main.js
├─ package.json
├─ webpack.config.js
├─ yarn.lock
└─ README.md
```

```
// Repo2: Shopping Cart
├─ assets
├─ components
│   └─ ShoppingCart.js
├─ node_modules
├─ pages
│   └─ main.js
├─ package.json
├─ webpack.config.js
├─ yarn.lock
└─ README.md
```

```
// Repo2: Inventory
├─ assets
├─ components
│   └─ InventoryComponent.js
├─ node_modules
├─ pages
│   └─ main.js
├─ package.json
├─ webpack.config.js
├─ yarn.lock
└─ README.md
```

Integrating GitFlow with DevOps

GitFlow Process with CI/CD

- CI: When a feature branch is completed and members are added, the programmer submits a pull request (PR) to merge with the developers. The CI process will automatically run to check the source code, including unit tests, security analysis, and security testing tools.
- CD: After the code has been merged into the development branch and checked through CI, the CD process deploys the code to the testing environment (staging), and then to the Production environment.

CI/CD Pipeline



GitHub Actions



Bitbucket Pipelines

GitHub Actions

GitHub Actions – Native CI/CD for GitHub

- Deep integration with GitHub
- YAML-based workflows (``.github/workflows/``)
- Supports self-hosted and cloud runners
- Free for public repositories



GitHub
Actions

GitLab CI/CD

GitLab CI/CD – Built-in CI/CD for GitLab Repos

- Easy integration with GitLab repositories
- YAML-based configuration (``.gitlab-ci.yml``)
- Supports auto-scaling runners
- Requires GitLab account



Bitbucket Pipelines

Built-in CI/CD for Bitbucket repositories

- YAML-based configuration (`bitbucket-pipelines.yml`)
- Easy integration with Bitbucket
- Requires a Bitbucket account



Bitbucket Pipelines

Thank you

5. GitHub Actions

— — —

Video: <https://www.youtube.com/watch?v=1oJQRlz1v94>